Understanding Gradient Flow in Sequential Models Through A Theoretical Reformulation of LSTM Memory Updates

Daniel Zeng

March 2025

1 Introduction

Modeling sequential data is an important problem in machine learning because of applications like natural language processing and time-series prediction. Traditional architectures such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were designed to address this challenge, but suffer from well-known limitations related to gradient flow over long sequences.

In this project, I first review the development of RNNs, LSTMs, and self-attention mechanisms with an emphasis on the mathematical causes of gradient flow issues. I then explore a novel reformulation of the LSTM cell state update as a softmax-like, attention-style weighted sum over past candidate memories. This perspective gives a new lens for understanding memory retention in recurrent architectures.

Through a series of small empirical experiments, I also visualize how memory contributions decay over time in LSTMs, and confirm the reformulation's correctness. The results reinforce the inherent limitations of gated recurrent structures, and suggest directions for future exploration of hybrid attention-inspired architectures.

2 Literature Review

2.1 Feedforward Models to Sequence Modeling

The development of neural networks has historically followed the structure of the data they were designed to model. Early innovations, such as the perceptron and multilayer perceptrons (MLPs), were the foundation for modern feedforward neural networks. These architectures were also adapted into Convolutional Neural Networks (CNNs) [1], which learned spatial hierarchies in image data.

However, many real-world problems, such as natural language processing, speech recognition, and time series forecasting, require models that can handle sequential data. These are data in a sequence where each observation may be conditional on prior data. Thus, a major limitation of feedforward architectures is their assumption of fixed-length inputs, while sequences often have variable lengths. It is difficult for standard MLPs to generalize across different sequence sizes or maintain temporal context. This observation motivated the development of models with internal "memory" capable of processing data one element at a time, while preserving context.

This led to the introduction of Recurrent Neural Networks (RNNs) [2], which process sequences recursively by maintaining a hidden state vector that captures information from previous time steps, which is used as input for the next element being processed. RNNs enabled models to handle sequences of arbitrary length by unrolling their computation over time and reusing the same weights at each step. Despite their conceptual simplicity, RNNs has flaws with gradient flow, especially as the sequence length grows.

2.2 Recurrent Neural Networks (RNNs)

2.2.1 Structure and Mathematical Formulation

A standard RNN includes hidden layer(s) whose outputs not only feed into forward layers, but also back into itself. At every iteration, this recurrent hidden layer takes in as input both the previous layer's output, and its own output from the previous iteration. The output *and* feedback loop of the recurrent layer can be thought of as recursively updating a hidden state vector $\mathbf{h}_t$ at each time step t using the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$. The output of a hidden layer is:

$$\mathbf{h}_t = \phi(\mathbf{W}_{hx}\mathbf{x}_t + (\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)), \tag{1}$$

where ϕ is a element-wise nonlinear activation function (typically tanh or ReLU), $\mathbf{W}_{hx}$, is the weight matrix for the non-recurrent input, $\mathbf{W}_{hh}$ is a weight matrix on the recurrent input, and $\mathbf{b}_h$ is a bias, all three of which are learnable parameters. The output *of the entire RNN* at time t is usually given by:

$$\mathbf{y}_t = \mathbf{W}_{qh}\mathbf{h}_t + \mathbf{b}_o. \tag{2}$$

To train RNNs, the network is "unrolled" over time and trained using Backpropagation Through Time (BPTT) [3]. This involves computing gradients of the loss with respect to parameters by recursively applying the chain rule over each time step.

2.2.2 Gradient Flow Problems During Training of RNNs

Suppose the RNN outputs a sequence of predictions $(\mathbf{y}_1, \ldots, \mathbf{y}_T)$ based on hidden states $(\mathbf{h}_1, \ldots, \mathbf{h}_T)$. The loss over the entire sequence is defined as:

$$\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t), \tag{3}$$

where ℓ is the per-step loss function. Using the chain rule, the gradient with respect to recurrent weights $\mathbf{W}_{hh}$ can be expressed as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \cdot \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}. \tag{4}$$

The challenging term is $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$, which depends recursively on all past time steps. The recurrence for this derivative unfolds as:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \frac{\partial f_t}{\partial \mathbf{W}_{hh}} + \frac{\partial f_t}{\partial \mathbf{h}_{t-1}} \cdot \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{W}_{hh}}, \tag{5}$$

where f_t denotes the hidden state update function at time t [4]. This recursion results in products of the form:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \sum_{k=1}^{t} \left(\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}} \right). \tag{6}$$

Each Jacobian $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ typically includes the derivative of the activation function (e.g., $\phi'(z) \leq 1$) and the weight matrix $\mathbf{W}_{hh}$. Over long sequences, repeated multiplication of such terms can cause the gradients to either:

- Vanish: if these products contain values less than 1, they shrink exponentially with each time step.
- Explode: if values exceed 1, they grow uncontrollably.

This may lead to numerical instability during training. Furthermore, important information from early time steps is either lost or causes unstable updates. These issues were analyzed by Bengio et al. [5], who showed that learning dependencies across more than 10–20 steps becomes unlikely using gradient descent.

Several strategies have been proposed to address this:

- Gradient Clipping: limit the norm of gradients to prevent explosion.
- Truncated BPTT: stop gradient backpropagation after a fixed number of steps.
- Randomized Truncation [6]: introduce randomization in truncation length to control the expectation values of gradients.

These techniques make RNN training more stable, but they also introduce bias toward short-term dependencies. This led to the development of architectures like LSTMs and Transformers that treat this limitation structurally.

2.2.3 Vanishing and Exploding Gradients from a Linear Algebra Point of View

Consider an RNN without inputs or nonlinear activation functions:

$$\mathbf{h}_t = \mathbf{W}^t \mathbf{h}_0. \tag{7}$$

If $\mathbf{W}$ is diagonalizable and has an eigendecomposition $\mathbf{W} = \mathbf{Q}\Lambda\mathbf{Q}^{-1}$, where Λ is a diagonal matrix of eigenvalues $(\lambda_1, \ldots, \lambda_n)$. Then the hidden state becomes:

$$\mathbf{h}_t = \mathbf{Q}\Lambda^t\mathbf{Q}^{-1}\mathbf{h}_0. \tag{8}$$

This linear algebraic perspective(introduced by Goodfellow et al.[7]) shows each component of the hidden state aligned with an eigenvector v_i is scaled by λ_i^t. If $|\lambda_i| < 1$, the contribution of that eigenvector gets smaller exponentially, leading to vanishing gradients. On the other hand, if $|\lambda_i| > 1$, that component grows rapidly, leading to exploding gradients.

Intuitively, each eigenvector direction defines a "mode" of memory retention. For large values of t, components of $\mathbf{h}_0$ aligned with the largest eigenvector persist, while all others vanish or explode depending on their corresponding eigenvalues.

2.3 Long Short-Term Memory (LSTM) Networks

The limitations of RNNs like their difficulties with long-range dependencies due to vanishing or exploding gradients led to the development of more complex architectures. The *Long Short-Term Memory* (LSTM) network was introduced by Hochreiter and Schmidhuber in 1997 [8]. The main idea behind LSTMs is to enable more reliable gradient flow across time steps, which is achieved through memory-control *gates* and a dedicated memory pathway called the *cell state*.

2.3.1 LSTM Architecture

In an LSTM, the hidden state from the previous time step $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$ are used to compute three gates and a candidate memory(cell state) value:

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{(input gate)} \tag{9}$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{(forget gate)} \tag{10}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{(output gate)} \tag{11}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_{xc}\mathbf{x}_t + \mathbf{W}_{hc}\mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{(candidate cell state)} \tag{12}$$

Here, σ denotes the element-wise sigmoid function and tanh is the hyperbolic tangent. All $\mathbf{W}$ weight matrices and $\mathbf{b}$ bias vectors are learnable parameters. The input gate, forget gate, and candidate cell state will be used to update the cell state (memory). Intuitively, the forget gate determines how much of previous memory to "forget", the input gate weights the importance of the new information to be added, and the candidate cell state is the new information. The output gate determines how much new information will contribute to changing the hidden state. The actual cell state $\mathbf{c}_t$ and hidden state $\mathbf{h}_t$ are then updated as:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{13}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{14}$$

where $\odot$ is element-wise multiplication. The key is the introduction of a linear self-loop in the memory cell state $\mathbf{c}_t$ through the forget gate $\mathbf{f}_t$. This allows information to be carried forward across many time steps with minimal interference (if the data encourages this) which allows the model to selectively store long-term information.

2.3.2 Improved Gradient Flow Through Time

LSTMs mitigate the vanishing gradient problem by providing paths through time that have more stable derivatives. As shown empirically by both Hochreiter and Schmidhuber [8] and later formalized by Gers et al. [9], the self-loop of the memory cell allows gradients to flow more directly.

To see this mathematically, consider the partial derivative of the loss $\mathcal{L}$ with respect to the cell state $\mathbf{c}_{t-k}$ at some earlier time step. Using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-k}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \prod_{j=t-k+1}^{t} \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \prod_{j=t-k+1}^{t} \mathbf{f}_j \tag{15}$$

Since each $\mathbf{f}_j \in (0, 1)$ is caclulated with a sigmoid, the cell state derivative is now a product of learned forget gate values instead of arbitrary nonlinearities in the data and recurrent weight matrices, which impacted RNNs. This allows the LSTM to learn to maintain or forget memory information explicitly due to increased model complexity. Unlike traditional RNNs, the magnitude of the gradient can remain close to one for significant time spans (if the data requires this), thus mitigating vanishing gradients and making long-term credit assignment feasible. This of course does not eliminate the possibility of vanishing/exploding gradients in extremely long sequences, but does allow for more flexibility in learning long dependencies than RNNs.

Intuitively, the forget gate allows the model to dynamically control the time constant of the memory cell: when $\mathbf{f}_t \approx 1$, the model "remembers" almost everything from before; when $\mathbf{f}_t \approx 0$, it "forgets" immediately.

2.3.3 Limitations of LSTM Networks

LSTMs are firstly computationally expensive because of the increased number of parameters required by their gating mechanisms. Second, while LSTMs reduce gradient problems, they do not completely eliminate them in very long sequences, as previously mentioned. Lastly, LSTMs still process sequences sequentially, which does not allow for parallelization.

2.3.4 Empirical Visualization of Vanishing Gradients

In this small experiment with random data sequence and arbitrary target values, gradient decay is noticeably slower with LSTMs, and the log scale figure shows the sequence lengths where gradients vanished.

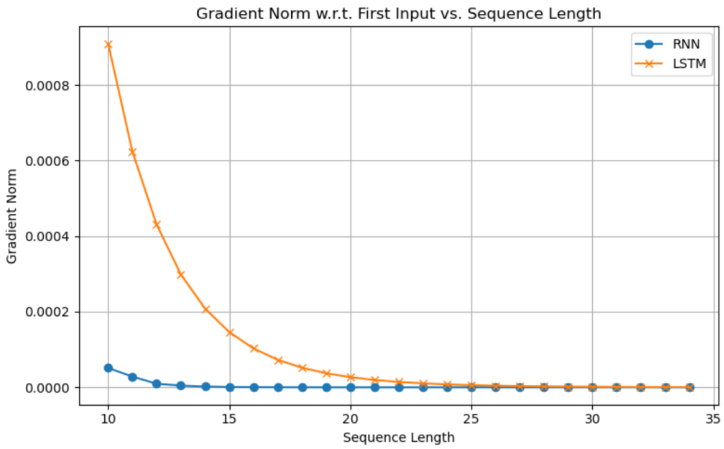


Figure 1: Gradient norm with respect to the first input vs. sequence length for RNN and LSTM models.

2.4 Self-Attention Mechanisms

While LSTMs were an improvement over traditional RNNs in addressing vanishing gradients and long-term dependencies, they still had a sequential processing nature. This prevents parallelism, and gradient propagation still degrades over very long sequences. Motivated by these challenges, the concept of *attention mechanisms* was developed.

Attention mechanisms were first introduced by Bahdanau et al. [10] to improve encoder–decoder RNN models for machine translation by allowing the decoder to dynamically focus on relevant parts of the input sequence at each output step. This eliminated the need to compress an entire input sequence's context into a single fixed-length vector (which is analogous to what we previously described as "memory"). The idea was later extended and redefined by Vaswani et al. [11] in their Transformer architecture, which relies entirely on self-attention to model dependencies between tokens (elements of a sequence) without requiring recurrence.

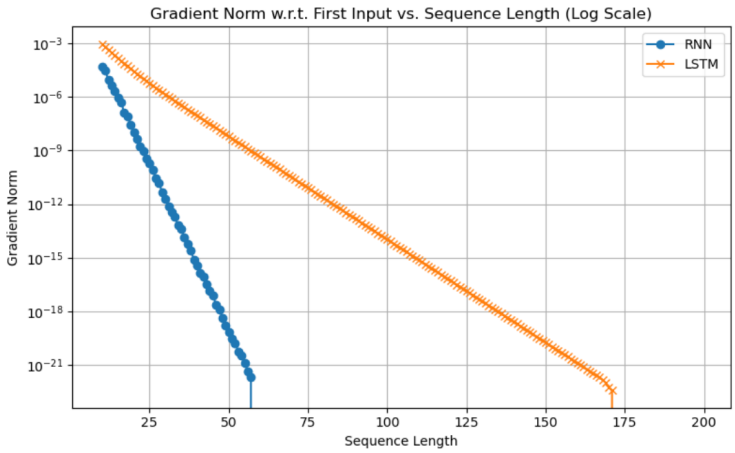


Figure 2: Same as figure 1, log scale used to show specific point at which gradient vanishes.

2.4.1 Queries, Keys, and Values in Self-Attention

At the heart of the attention mechanism lies the notion of *query*, *key*, and *value* vectors. Intuitively, these can be thought of as follows: a **query** represents the element seeking information, a **key** represents the label or identity of available information, and a **value** represents the actual content or data associated with each key. In the context of self-attention, all three are derived from the same input sequence, but are linearly projected into separate spaces.

The attention mechanism computes a weighted sum of value vectors $\mathbf{v}_j$, where the weights depend on how well a query $\mathbf{q}_i$ "matches" or aligns with each key $\mathbf{k}_j$. This match is quantified by a score function, typically chosen as the (scaled) dot product between the query and key:

$$\text{score}(\mathbf{q}_i, \mathbf{k}_j) = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}}, \tag{16}$$

where d is the dimensionality of the key and query vectors. This score can be thought of as a similarity or alignment measure—larger values indicate a stronger match between the query and key. The division by $\sqrt{d}$ is for normalization: as the dimensionality increases, the dot product magnitude can grow too large, pushing the softmax into regions with very small gradients. Dividing by $\sqrt{d}$ improves numerically stability. [11].

To convert these scores into attention weights α_{ij} that sum to 1 and can be used for weighted averaging, a *softmax* function is applied. This is done in three conceptual steps:

1. First, the raw scores are exponentiated to guarantee all positive values. This also furthers the difference between high and low scores:
$$\exp(\text{score}(\mathbf{q}_i, \mathbf{k}_j))$$

2. Next, each exponentiated score is normalized by dividing it by the sum of all exponentiated scores over the keys. This creates a valid probability distribution representing the "probability" or "similarity" of the query matching any of the key values:
$$\alpha_{ij} = \frac{\exp(\text{score}(\mathbf{q}_i, \mathbf{k}_j))}{\sum_{l=1}^{n} \exp(\text{score}(\mathbf{q}_i, \mathbf{k}_l))}$$

3. The resulting weights $\alpha_{ij} \in (0, 1)$ add to 1 across all j keys. This makes them sufficient for creating a convex combination of the value vectors.

Finally, the attention score for a particular query $\mathbf{q}_i$ is a weighted sum over the values:

$$\text{Attention}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) = \sum_{j=1}^{n} \alpha_{ij} \mathbf{v}_j. \tag{17}$$

This process allows each position in the input sequence to selectively "attend" to other positions(queries), depending on the learned relationships encoded in the queries and keys. Unlike in RNNs or LSTMs, this attention is not sequential/temporal. Self-attention allow modeling of dependencies forward and backward in the same sequence directly, which includes long-range dependencies.

In matrix form, we denote the stacked queries, keys, and values as $\mathbf{Q} \in R^{n \times d}$, $\mathbf{K} \in R^{n \times d}$, and $\mathbf{V} \in R^{n \times d_v}$. The attention output is:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}. \tag{18}$$

This function is differentiable and allows attention scores to be calculated across the entire sequence. Importantly, in *self-attention*, the queries, keys, and values all come from the same input sequence itself.

2.4.2 Self-Attention in the Transformer Architecture

Though my focus is on attention scores, it is useful to explain them briefly within the Transformer architecture. In the full architecture, the attention mechanism is applied in parallel across all input positions which creates contextualized representations. These attention outputs are typically combined with other feedforward layers, layer normalization, and other connections to form transformer blocks [11].

2.4.3 Why Attention Mechanisms Solve Long-Term Dependency Issues

One of the most significant advantages of self-attention over RNN-based models is its ability to capture *global dependencies* without requiring information to be passed through a chain of recurrent steps. In RNNs, information about a token at time t must propagate through all subsequent time steps to influence token $t + k$, leading to gradients that may vanish or explode. In contrast, attention scores are computed directly between any pair of positions, regardless of their distance.

Mathematically, consider the gradient of the attention output with respect to an input vector $\mathbf{x}_j$. Because attention uses dot-products and softmax operations, the gradient signal is distributed across all positions simultaneously:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_j} = \sum_{i=1}^{n} \frac{\partial \mathcal{L}}{\partial \alpha_{ij}} \cdot \frac{\partial \alpha_{ij}}{\partial \mathbf{x}_j} + \frac{\partial \mathcal{L}}{\partial \mathbf{v}_j} \cdot \frac{\partial \mathbf{v}_j}{\partial \mathbf{x}_j}, \tag{19}$$

where α_{ij} includes terms from all dot-products $\mathbf{q}_i^\top \mathbf{k}_j$. Unlike in RNNs or LSTMs, there is no dependency on sequential gradient chains, and therefore no risk of exponential decay of gradients across time.

Additionally, because attention weights form a convex combination, they are bounded in magnitude:

$$\sum_{j=1}^{n} \alpha_{ij} = 1, \quad \alpha_{ij} \geq 0, \tag{20}$$

2.4.4 Empirical Visualization of Long-Term Dependency Attention Scores in Self-Attention

In this small experiment with random data, I plotted attention scores between query and key values. Squares near the diagonal are more "similar" or equal query and keys. It is clear that there is no correlation between similarity and higher attention scores. This shows the ability for self-attention to model long-term dependencies.

2.5 Attention-like Interpretation of LSTM Cell Updates

Although LSTMs and Transformer-based models both try to capture dependencies in sequential data, they do so through different methods: recurrence versus global weighting. In this section, I explore a mathematical perspective that reinterprets the LSTM cell update as an attention-like weighted sum, with the goal of connecting gating mechanisms in recurrent architectures to the softmax-based attention weights used in Transformers.

2.5.1 Unrolling the LSTM Cell Update

Beginning with the LSTM cell state update equation:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \tag{21}$$

where f_t is the forget gate, i_t is the input gate, $\tilde{C}_t$ is the candidate memory content, and all operations are elementwise. The gate activations are computed as:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f), \quad i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i). \tag{22}$$

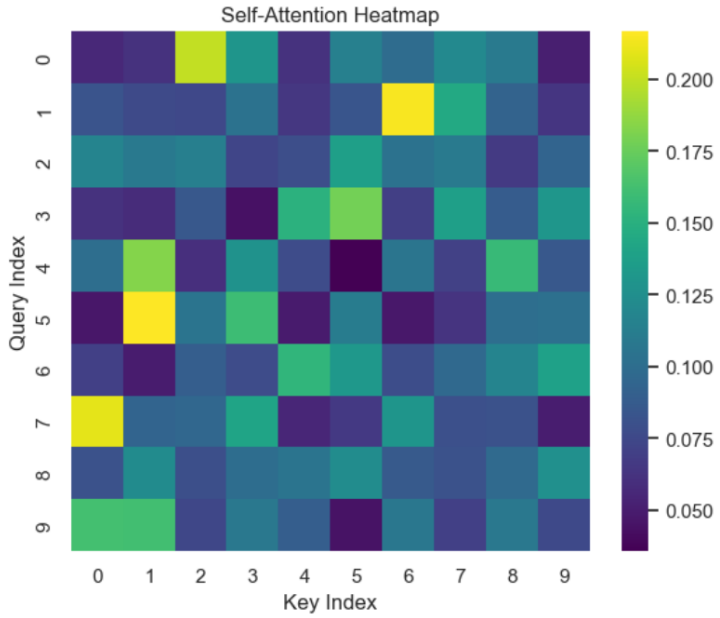


Figure 3: Heatmap of attention scores from self-attention on small random data sequence.

To understand how past candidate cell states contribute to the current state, I recursively expand the recurrence and solve by substitution.

First, substitute the expression for C_{t-1}:

$$C_{t-1} = f_{t-1} \odot C_{t-2} + i_{t-1} \odot \tilde{C}_{t-1}. \tag{23}$$

Substituting this into C_t:

$$\begin{aligned} C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\ &= f_t \odot \left(f_{t-1} \odot C_{t-2} + i_{t-1} \odot \tilde{C}_{t-1} \right) + i_t \odot \tilde{C}_t \\ &= f_t \odot f_{t-1} \odot C_{t-2} + f_t \odot i_{t-1} \odot \tilde{C}_{t-1} + i_t \odot \tilde{C}_t. \end{aligned} \tag{24}$$

Now substitute another step backwards with C_{t-2}:

$$C_{t-2} = f_{t-2} \odot C_{t-3} + i_{t-2} \odot \tilde{C}_{t-2}. \tag{25}$$

Plugging this in gives:

$$\begin{aligned} C_t &= f_t \odot f_{t-1} \odot \left(f_{t-2} \odot C_{t-3} + i_{t-2} \odot \tilde{C}_{t-2} \right) + f_t \odot i_{t-1} \odot \tilde{C}_{t-1} + i_t \odot \tilde{C}_t \\ &= f_t \odot f_{t-1} \odot f_{t-2} \odot C_{t-3} + f_t \odot f_{t-1} \odot i_{t-2} \odot \tilde{C}_{t-2} \\ &\quad + f_t \odot i_{t-1} \odot \tilde{C}_{t-1} + i_t \odot \tilde{C}_t. \end{aligned} \tag{26}$$

Continuing this recursive expansion, a pattern is observed in which each past candidate cell state $\tilde{C}_k$ is multiplied by its corresponding input gate i_k, and then scaled by the *element-wise product* of all forget gates from $j = k+1$ to t. The result is:

$$C_t = \sum_{k=1}^{t} \left(\prod_{j=k+1}^{t} f_j \right) \odot i_k \odot \tilde{C}_k. \tag{27}$$

*Note that for $k = t$, the product $\prod_{j=t+1}^{t} f_j$ is the empty product, which equals a vector of 1s. Thus, the final term reduces to:

$$\left(\prod_{j=t+1}^{t} f_j\right) \odot i_t \odot \tilde{C}_t = i_t \odot \tilde{C}_t,$$

which matches the original recurrence.

*Note also that all product(pi) notations in this section, exponentiation functions, log functions, and division (seen below with the softmax equation in (34)) are considered element-wise, so as to be correct per-dimension of the cell state *at a particular time*.

Interpretation :

The equation (27) shows that the LSTM cell state is effectively a weighted sum over past candidate states $\tilde{C}_k$, where the weight assigned to each $\tilde{C}_k$ is:

$$\alpha_k = \left(\prod_{j=k+1}^{t} f_j\right) \odot i_k. \tag{28}$$

This form resembles the structure of attention weights.

Here, the multiplicative term

$$\alpha_k = \left(\prod_{j=k+1}^{t} f_j\right) \odot i_k \tag{29}$$

acts as a weight assigned to each candidate memory $\tilde{C}_k$. It represents the accumulated effect of input and forget gates from step k to the present time t.

2.5.2 Linking to Attention Weights

In self-attention mechanisms, attention weights over past inputs are calculated via softmax normalization of pairwise scores. A typical formulation for the attention weight α_k^{attn} is:

$$\alpha_k^{\text{attn}} = \frac{\exp(q_k)}{\sum_{l=1}^{t} \exp(q_l)}, \tag{30}$$

where q_k is the scalar compatibility score between the query and the kth key. To align the LSTM formulation with this, we take the logarithm of the product over forget gates:

$$\log\left(\prod_{j=k+1}^{t} f_j\right) = \sum_{j=k+1}^{t} \log f_j. \tag{31}$$

We define a new quantity q_k, resembling an attention score, from equation (29) as:

$$q_k = \sum_{j=k+1}^{t} \log f_j + \log i_k, \tag{32}$$

so that:

$$\alpha_k \approx \exp(q_k). \tag{33}$$

To obtain attention-like weights, we normalize these scores across all k:

$$\alpha_k^{\text{LSTM}} = \frac{\exp(q_k)}{\sum_{l=1}^{t} \exp(q_l)}. \tag{34}$$

With this substitution, we can rewrite the cell state as:

$$C_t \approx \sum_{k=1}^{t} \alpha_k^{\text{LSTM}} \odot \tilde{C}_k, \tag{35}$$

which takes the form of an attention-style weighted sum over previous candidate cell states.

2.5.3 Interpretation and Assumptions

This (very convoluted) reformulation shows that the cumulative effect of the LSTM's gates resembles a compatibility or "similarity"-based weighting mechanism over the segment of input sequence *prior to the current time* t. The derivation makes several simplifying assumptions:

- The derivation treats gate activations f_j, i_k, and memory contents $\tilde{C}_k$ as elementwise vectors, and all operations/functions as elementwise, which ignors potential interdependencies across dimensions.
- For simplicity, linear transformations, biases, and nonlinearities in the gating equations were not shown, and assumed to be present in the score definition q_k.
- The reformulation is approximate and meant to show similarities rather than equivalence.

Notably, a big difference remains: in LSTMs, the aggregation is inherently causal and temporal, as only prior inputs affect the current state, whereas self-attention includes all tokens in the sequence, including future ones. Further, the softmax-normalization here only applies to previous dependencies *per time step*, and is not standard across the entire sequence.

2.5.4 Empirical Visualizations of LSTM Attention-like Weights and Gradient Flow

To further explore the behavior of the LSTM cell state reformulation as an attention-like mechanism, I conducted small numerical experiments to visualize the distribution of the effective weighting terms α_k over past timesteps. These plots try to show how much memory contribution is retained from earlier versus later timesteps, as well as how the design of forget gates influences gradient flow.

I first plotted the average value of the derived attention-like weights α_k as a function of timestep k, across a random LSTM sequence. As shown in Figure 4, the weights are heavily biased toward the most recent timesteps. As expected, older candidate states have very small associated weights, suggesting that the LSTM preferentially retains more recent information. This confirms that even with improved memory control via gates, LSTMs still struggle to retain information from distant past events.

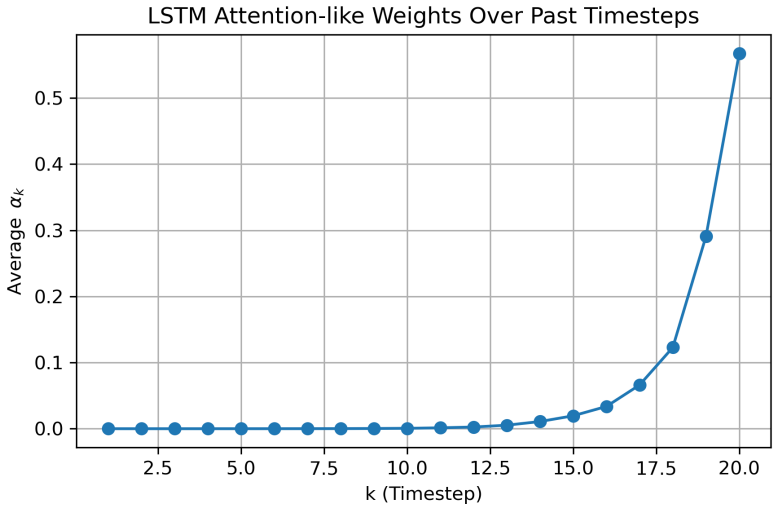


Figure 4: Average attention-like weights α_k as a function of timestep k in a random LSTM sequence.

Next, I visualized the attention-like weights per hidden unit across time using a heatmap (Figure 5). This allows inspection of how different hidden dimensions vary in their memory patterns. Again, later timesteps dominate the memory contributions, while earlier inputs have very little influence across most hidden units.

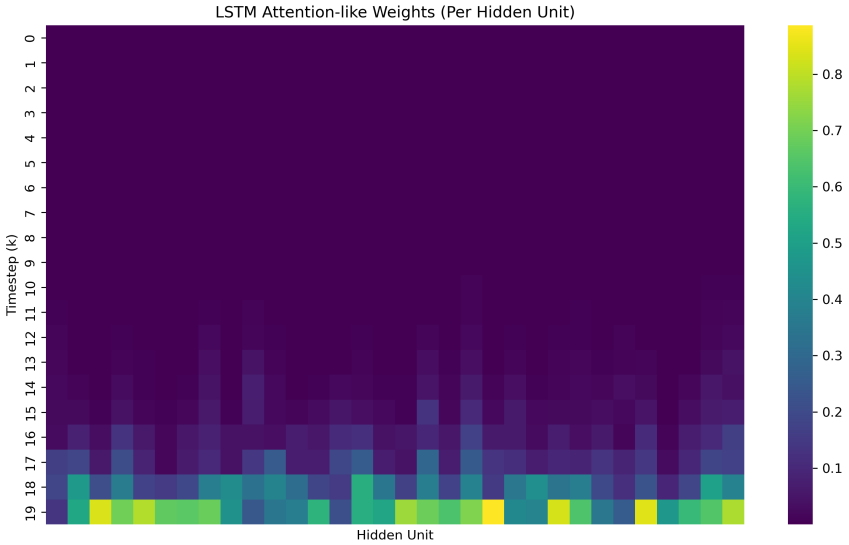


Figure 5: Heatmap of attention-like weights α_k across hidden units and timesteps. Brighter colors indicate stronger contribution to the current memory.

Finally, I performed a small comparative experiment to study the effect of forget gate biasing. I trained two small LSTMs on a toy sequence task: one with normal initialization, and one with the forget gate biases initialized toward larger values (favoring memory retention). As shown in Figure 6, the model with biased forget gates maintained significantly larger gradient norms with respect to the first input over the course of training. However, over time this does not necessarily improve or worsen first input gradient. This result supports the theoretical view that careful forget gate initialization can "encourage" long-term dependency learning by constant factors, but are still limited asymptotically over time.

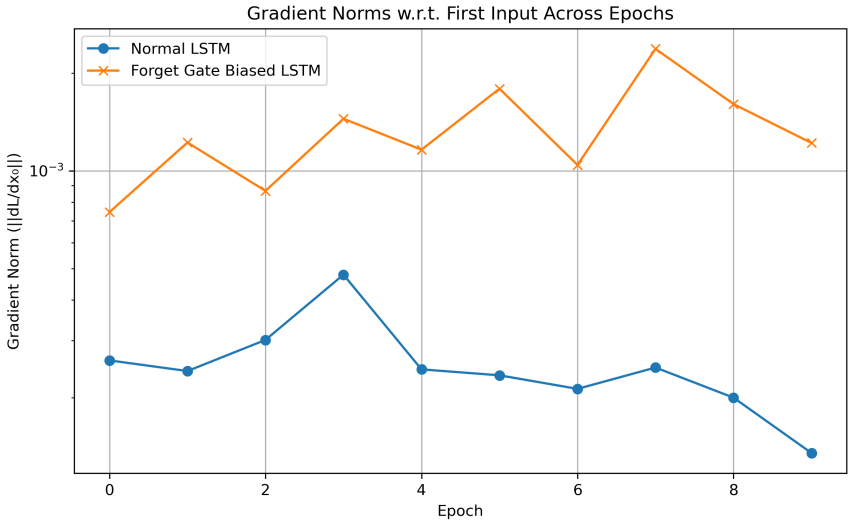


Figure 6: Gradient norms with respect to the first input across epochs for a normal LSTM and a forget-gate-biased LSTM. Forget gate bias improves gradient flow.

Overall, these visualizations show that the interpretation of the LSTM cell update as an attention-like aggregation display the same limitations as the regular LSTM, which supports the correctness of the reinterpretation. Furthermore, forget gate biasing may be an effective but non-proportional improvement over time.

2.5.5 Possible Implications of This Analysis

Although the reformulation does not change the computation of the LSTM, it may provide a new perspective to understand memory retention. This has several potential benefits:

First, the softmax-normalized form provides clearer interpretability by providing a way to find which candidate memories $\tilde{C}_k$ have the greatest impact on the current cell state without backtracking all the way. This contrasts with standard LSTMs where the influence of past inputs is less explicit.

Second, the formulation may help with theoretical analysis of gradient flow, by treating memory contributions as a weighted sum rather than a deep product of gates.

Finally, the relationship between recurrence and self-attention may help with new hybrid architectures that combine causal/temporal structure with attention-like weighting, if there exists some sequential task requiring more interpretability.

3 Conclusion

In this project, I reviewed the historical development of neural architectures for sequential modeling, including RNNs, LSTMs, and self-attention mechanisms. I studied the gradient flow issues that affect training stability. Then, I was able to reformulate the LSTM cell update equation into a self-attention-like weighted sum to show that the LSTM memory operates like a "recursive" self-attention mechanism. Through small numerical experiments and visualizations, I confirmed that although LSTM memory control improves over RNNs, long-term dependency modeling remains limited, as evidenced by the decay of effective memory contributions over time. Biasing forget gates toward memory retention improved gradient flow initially, but did not eliminate asymptotic decay. These findings support both the strengths and the limits of LSTM-like architectures and motivate further exploration of hybrid or attention-inspired memory mechanisms for sequential data.

3.1 Future Work

While several small experiments and reformulations were conducted in this project, there remain several possible avenues for future work:

1. Theoretical Analysis of Gradient Flow: While numerical simulations demonstrated improved gradient retention when biasing forget gates, a formal theoretical analysis could be done. Specifically, bounding the gradient contributions of each past candidate memory state under the reformulated attention-like structure could provide stronger guarantees about gradient sizes over time.

2. Quantitative Comparison of Memory Retention: Using the softmax-normalized interpretation of memory retention, future work could compute and visualize the weight α_k^{LSTM} for each candidate $\tilde{C}_k$ across time steps in real sequential tasks. This would quantify how much different memories contribute to the current cell state, and compare this weighting behavior between standard LSTM updates and attention-based models.

3. Topological Ordering of Attention Weight Graph: Interpret the attention weights as a weighted, directed graph where each node corresponds to a token and the edge weight reflects its attention-based influence. An issue that may come up are cyclic dependencies in the graph. Perhaps one could find a way to remove or collapse weighted cycles—by, for example, retaining only "forward" edges (i.e., for $i < j$) or applying a condensation algorithm. Then, we can get a weighted directed acyclic graph (DAG) where a topological ordering of tokens can be derived. What would happen if we then put this topological ordering of tokens through recursive model like RNN or LSTM, instead of a transformer?

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," in *Proceedings of the IEEE*, vol. 86, 1998, pp. 2278–2324.

[2] J. L. Elman, "Finding structure in time," *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.

[3] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.

[4] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. Cambridge University Press, 2023, `https://d2l.ai/`.

[5] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[6] C. Tallec and Y. Ollivier, "Unbiasing truncated backpropagation through time," *arXiv preprint arXiv:1705.08209*, 2017.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016, ISBN: 9780262035613. [Online]. Available: `https://www.deeplearningbook.org/`.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with lstm," *Neural computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

[10] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[11] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.